

Agent-Friendly by Design: An Empirical Comparison of MoonBit and Rust in AI-Driven Software Construction

[Author names]

[Affiliations]

Abstract

AI coding agents now write substantial amounts of production code, which makes the choice of implementation language an engineering variable with a measurable cost: every token the agent reads or writes is paid for, and every compiler round trip adds latency. We ask whether a young language designed with tooling and conciseness in mind can compete with an established systems language when an agent, not a human, does the programming. We had Claude Opus 5.5, driven by Claude Code, build the same SQLite-compatible SQL engine three times in Rust and three times in MoonBit. Each build ran nine milestones and ended at about 13,000 lines, and was graded against 489 conformance tests generated from SQLite 3.53, half of them hidden from the agent. Despite a far smaller presence in the model's training data, MoonBit reached parity: mean estimated agent cost was $0.94\times$ Rust's (median $1.12\times$), hidden-test pass rates were 95.6% versus 96.3%, and wall-clock time was indistinguishable. MoonBit programs were $0.88\times$ the size of Rust programs in model tokens, and agents wrote $0.88\times$ as many output tokens. We trace these savings to specific design features (garbage collection without ownership annotations, effect-typed errors, package-level namespaces) and trace MoonBit's remaining costs to fixable causes: a language primer re-read on every call, a small set of recurring syntax errors, and gaps in the core library. We derive design principles for agent-oriented languages and report the first of the resulting library improvements.

Keywords: AI coding agents, programming language design, token efficiency, MoonBit, Rust, large language models, empirical software engineering

1 Introduction

Large language models have moved from completing single functions to carrying out multi-hour software projects as autonomous agents. In that setting, cost scales with tokens: the agent re-reads its working context on every model call and writes code, reasoning and commands as output. Two properties of the implementation language therefore matter directly. First, how many tokens a feature takes to express, because that code is written once and re-read many times. Second, how often the agent's first attempt fails to compile, because each failure costs another round trip with a large context.

These properties pull in different directions for a new language. Familiar languages benefit from enormous training corpora, so models rarely make syntax or API mistakes in them. A newer language may be designed to be more concise and more regular, but models know it poorly. Existing code-generation benchmarks mostly measure function-level correctness across languages [1, 2] or issue resolution in existing repositories [3]. Neither isolates the effect of the language on the cost of building a sizable system from scratch.

We study this question with MoonBit, a statically typed language targeting WebAssembly, JavaScript and native code, whose design and toolchain were built with AI-assisted development in mind [4]. We compare it with Rust, a mature language with a comparable type system (algebraic data types, pattern

matching, traits, no null) and extensive training data. Holding everything but the language fixed, we measure what an agent spends to build the same non-trivial program in each.

Our contributions are:

- A controlled, reproducible benchmark: a nine-milestone SQLite-compatible database engine with 489 SQLite-generated conformance tests, identical scaffolding for both languages, and an orchestration harness that records every token, tool call and test result.
- Evidence from six complete builds that MoonBit reaches cost and quality parity with Rust for an AI agent, while producing about 12% smaller programs.
- A decomposition of MoonBit's remaining overhead into its causes, each with measured cost, and a set of design principles for agent-oriented languages.
- The first library change prompted by these findings: ordered-map navigation in MoonBit's core library, currently under review.

2 Agent-Relevant Features of MoonBit

MoonBit shares much of Rust's surface: expression-oriented syntax, enums with payloads, exhaustive pattern matching, traits, generics and `Option` types. It differs in several ways we expected to matter for agents:

- **Garbage collection without ownership.** Values are shared by reference, and there are no lifetimes, borrows or clones to write. Rust expresses the same aliasing explicitly (`&`, `&mut`, `Box`, `Rc<RefCell<·>>`), which costs tokens and produces a class of compile errors with no MoonBit counterpart.
- **Effect-typed errors.** A function declares `raise E` in its signature and errors propagate through plain calls. Rust marks each propagation site with `?` and wraps results in `Result`, `Ok` and `Err`.
- **Packages as shared namespaces.** All files in a directory form one compilation unit with no per-file imports or visibility annotations between them.
- **Fast, queryable tooling.** `moon check` type-checks a 13,000-line project in about 0.2 s, and `moon ide doc` answers API questions from the command line.
- **Different defaults.** Strings are UTF-16, `Int` is 32-bit, and integer arithmetic wraps. These are reasonable design choices, and UTF-16 in particular suits the language's JavaScript and WebAssembly targets. They do, however, differ from what a Rust-trained model expects.

From these we formed four hypotheses. **H1:** MoonBit programs need fewer tokens for the same behavior. **H2:** the agent's unfamiliarity with MoonBit costs extra compile failures, but that cost is bounded. **H3:** total agent cost is comparable between the languages. **H4:** quality is comparable.

3 Methodology

3.1 Task

The agent builds *minisql*, a command-line program that executes a SQL script from standard input and prints results, matching SQLite 3.53. We chose this task because the reference implementation doubles as an exact oracle, because the work decomposes into well-defined stages, and because it spans parsing, semantic analysis, query planning, execution and a binary file format. Table 1 lists the milestones.

Table 1. Milestones and conformance tests. Expected outputs were generated by SQLite; each test was rerun with `PRAGMA reverse_unordered_selects` and a top-level `ORDER BY` check to reject order-dependent cases. Half of each milestone’s tests, chosen by hash, were withheld from the agent.

Milestone	Scope	Tests
M1 Core pipeline	Lexer, parser, CREATE/INSERT/SELECT, type affinity, output protocol	53
M2 Expressions	CASE, CAST, LIKE/GLOB, collations, ~60 scalar and math functions, printf	68
M3 DML and constraints	ORDER BY/LIMIT, UPDATE/DELETE, conflict clauses, upsert, RETURNING	56
M4 Aggregation	Aggregates, GROUP BY/HAVING, date and time functions	53
M5 Joins and subqueries	All join types, correlated subqueries, compound queries	56
M6 Schema and transactions	Indexes, views, ALTER TABLE, savepoints, performance tests	55
M7 CTEs and windows	Recursive CTEs, window functions with all frame types	61
M8 Read SQLite files	SQLite database file format: b-trees, records, overflow pages	42
M9 Write SQLite files	Files accepted by SQLite’s <code>integrity_check</code>	45
Total		489

3.2 Agent and harness

Each milestone ran as a fresh non-interactive Claude Code session (version 2.1.281) using Claude Opus 5.5 at high reasoning effort, with only the Bash, Read, Edit and Write tools. User settings, memory, skills, plug-in servers and web access were disabled. A fresh session per milestone means the cost of re-reading the growing codebase is part of the measurement. If visible tests still failed when the agent stopped, the harness could send up to two follow-up prompts; none were needed. After each milestone the harness committed a snapshot, scored it on pristine copies of the visible and hidden tests, measured the source, and checked compliance.

3.3 Fairness controls

- Identical specification, tests, prompts, model, effort level and tools for both languages.
- Standard library only, with no third-party packages, foreign-function calls, C code or embedded SQLite; verified automatically after every milestone.
- An identical five-function I/O module (read stdin, write stdout, arguments, read and write whole files) in both starter projects, because MoonBit’s core library has no file or stdin access.
- A language primer in each workspace with the same four sections: toolchain, language essentials, core-library reference and pitfalls. MoonBit’s (11.7k tokens) is longer than Rust’s (2.9k) because the model knows MoonBit less well. Every snippet was compiled against the installed toolchains, and neither primer contains advice about building databases.
- Rust and MoonBit trials ran concurrently in pairs so both saw the same service conditions.

3.4 Metrics

From the recorded session streams we take input tokens (split into cache reads and writes), output tokens (including reasoning), API calls, turns and wall time. Cost is Claude Code’s estimate at API list prices; a linear fit over all 54 sessions reproduces it to within \$0.07 per session with effective prices of \$0.20, \$8.05 and \$19.92 per million cache-read, cache-write and output tokens. We also record input tokens until the visible tests first all pass, which separates the cost of reaching a working implementation from optional hardening afterwards. Source size is measured in the model’s own tokenizer.

4 Results

All six builds completed all nine milestones without follow-up prompts, rule violations or access to the hidden tests. Table 2 summarizes the per-trial totals.

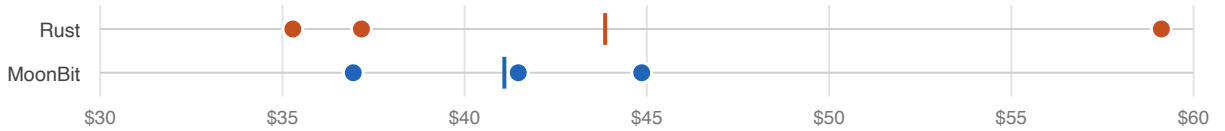
Table 2. Per-trial totals over nine milestones, three trials per language. Ratios are MoonBit divided by Rust.

Metric (per trial)	Rust mean $\pm$ sd	MoonBit mean $\pm$ sd	Ratio	Median ratio
Estimated cost (USD)	43.86 $\pm$ 13.25	41.09 $\pm$ 3.97	0.94	1.12
Output tokens	868k $\pm$ 190k	762k $\pm$ 45k	0.88	0.97
Total input tokens	68.9M $\pm$ 38.3M	67.8M $\pm$ 15.0M	0.98	1.45
Input until first all-green	21.7M $\pm$ 1.7M	23.6M $\pm$ 1.3M	1.09	1.11
Primer re-reads	1.1M $\pm$ 0.2M	4.9M $\pm$ 0.4M	4.29	4.71
API calls	398 $\pm$ 63	420 $\pm$ 36	1.06	1.16
Wall time (h)	2.40 $\pm$ 0.41	2.21 $\pm$ 0.12	0.92	1.00
Failed builds	5.3 $\pm$ 1.5	16.0 $\pm$ 2.6	3.00	3.00
Hidden tests passed (of 242)	233.0 $\pm$ 2.6	231.3 $\pm$ 0.6	0.99	0.99
Final source tokens	194k $\pm$ 20k	172k $\pm$ 5k	0.88	0.93
Final code lines	13,298 $\pm$ 1,217	13,842 $\pm$ 383	1.04	1.00

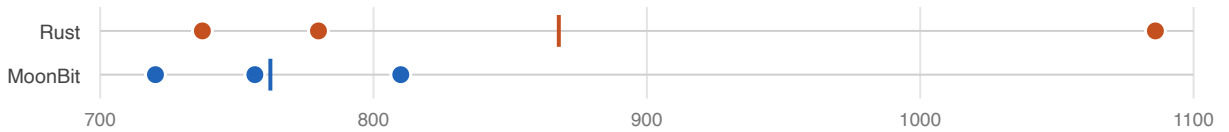
4.1 Cost and quality are at parity (H3, H4)

Mean estimated cost favored MoonBit (0.94 $\times$) while the median favored Rust (1.12 $\times$); the difference in either direction is smaller than the spread within a language (Figure 1). Rust's costs ranged from \$35.29 to \$59.12 and MoonBit's from \$36.94 to \$44.86. Hidden-test pass rates were 96.3% for Rust and 95.6% for MoonBit, about two tests of 242 per trial. We therefore read the data as parity on both cost and quality. Three trials per language cannot resolve a difference of a few percent.

Estimated cost (USD)



Output tokens (thousands)



Input until first all-green (millions)

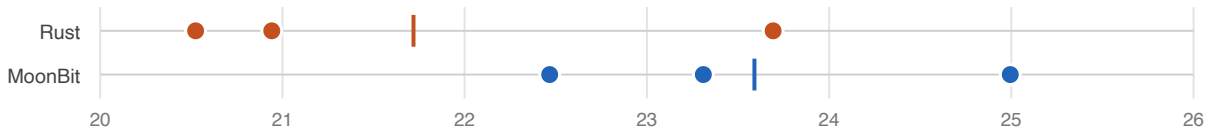


Figure 1. Per-trial results. Dots are individual nine-milestone builds; ticks are means. The spread within each language is comparable to the gap between languages.

4.2 Agents spend most tokens after the tests pass

The largest source of variance was not the language. In every build, agents kept working after their visible tests first all passed: fuzzing their engine against SQLite, profiling, and handling edge cases on their

own initiative. Between 47% and 82% of each build's input tokens came after that point (Figure 2). Rust's most expensive build and MoonBit's second build each contained one such extended session. Measured only up to the first all-green point, MoonBit used $1.09\times$ Rust's input tokens; the larger primer, re-read on every call, accounts for about 2.1M tokens per trial, which covers essentially all of that difference.

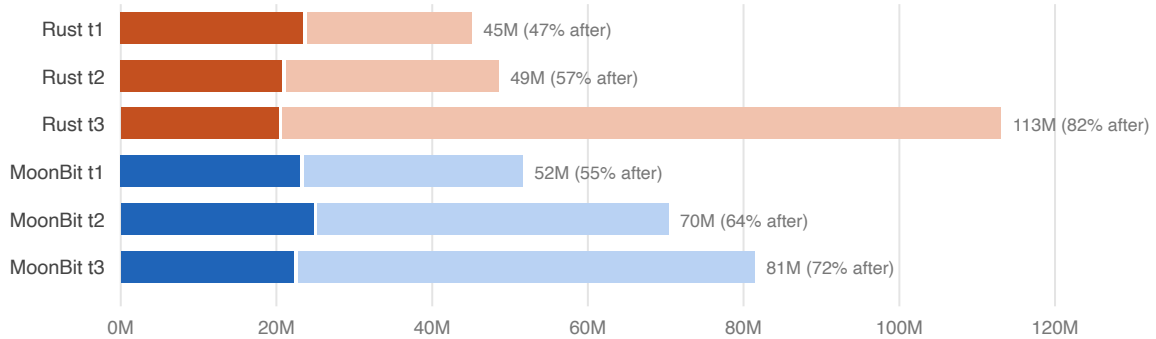


Figure 2. Input tokens per build, split at the moment all visible tests first passed (dark) and afterwards (light).

4.3 MoonBit programs are smaller (H1)

Agents wrote $0.88\times$ as many output tokens in MoonBit, and the finished MoonBit engines were $0.88\times$ the size of the Rust engines in model tokens, with about the same number of lines. MoonBit used fewer output tokens on eight of nine milestones (Figure 3). Comparing the same component across one pair of builds shows where the difference comes from (Table 3). The SQLite file reader, which is dominated by error handling and byte access, was 39% smaller in MoonBit: raise in 12 signatures replaced 14 ? operators, 12 Ok/Err constructors, 7 Result types, 43 borrows and 7 lifetimes. Components dominated by string and number handling were larger in MoonBit, because the agents converted between UTF-16 code units and characters and wrote numeric-formatting code that Rust's standard library provides.

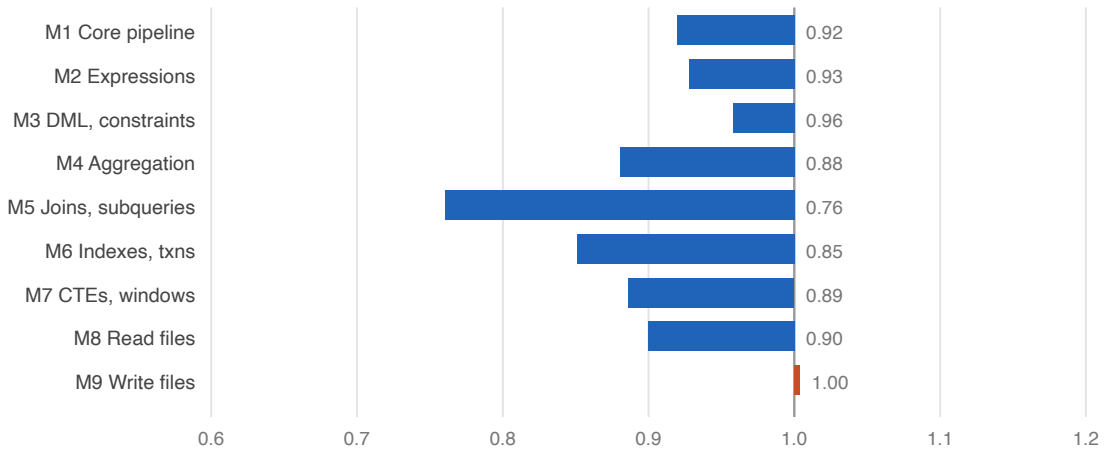


Figure 3. Output tokens per milestone, MoonBit divided by Rust, mean of three builds. Bars left of 1.0 mean MoonBit wrote less.

Table 3. The same component in one pair of builds (trial 1). Tokens approximated as characters/4 of non-comment code.

Component	Rust lines / tokens	MoonBit lines / tokens	Token ratio
SQLite file reader	317 / 2,964	279 / 1,808	0.61
Parser	1,536 / 13,835	1,806 / 11,328	0.82
Window functions	707 / 6,330	845 / 5,172	0.82
File writer	483 / 4,540	615 / 3,784	0.83
Date and time	796 / 6,426	951 / 5,370	0.84
Lexer	259 / 2,294	349 / 2,194	0.96
printf	436 / 3,243	622 / 3,629	1.12
Values and operators	511 / 3,785	860 / 4,482	1.18

4.4 Unfamiliarity costs compile errors, not quality (H2)

MoonBit builds failed 13.0% of the time against 5.8% for Rust. Hand-labelling every failure shows that about half of MoonBit's were language-neutral (unfinished code checked mid-edit, or ripple effects of a refactor), because the primer recommended type-checking after every edit and a check took only 0.2 s. The language-specific failures (Table 4) were concentrated in a few constructs the primer did not mention. Across three builds they cost about 44 API calls, against 12 for Rust's borrow-checker and type-inference errors, a net of roughly 11 calls and \$0.43 per build. The primer's list of removed syntax was effective: none of the six obsolete forms it named appeared in any diagnostic.

Table 4. Language-specific failed builds by primary cause, all three builds per language.

Cause	Failed builds	Builds affected
MoonBit: <code>using</code> is a keyword; renames prompted by reserved-word warnings	6	3 of 3
MoonBit: <code>mut</code> inside a tuple pattern, <code>let (mut a, b) = ...</code>	4	2 of 3
MoonBit: error hidden when output was filtered with <code>tail</code>	4	2 of 3
MoonBit: non-raising function where a raising function type is expected	2	3 of 3
MoonBit: <code>C(..)</code> on a positional constructor	2	1 of 3
MoonBit: invented core API; other syntax	7	3 of 3
Rust: borrow checker (4), type annotations (1), immutable binding (1)	6	3 of 3

4.5 Time and runtime

Wall time was the same (median 2.19 h per build in both languages). About 93% of it was model generation, so build speed had little influence: builds and tests took 8 to 14 minutes per build. On the finished engines MoonBit's native release build was slower (7 to 9 s against 3 to 4 s), while its type check was faster (0.2 s against about 1 s). On ten controlled workloads the MoonBit engines ran at 1.25× the time of the Rust engines, faster on string functions and slower on index construction and sorting (Figure 4). Profiling traced most of that gap to the ordered map the MoonBit agents had to write themselves (Section 5). Peak memory on the index workload was 170 MB for MoonBit against 601 MB for Rust.

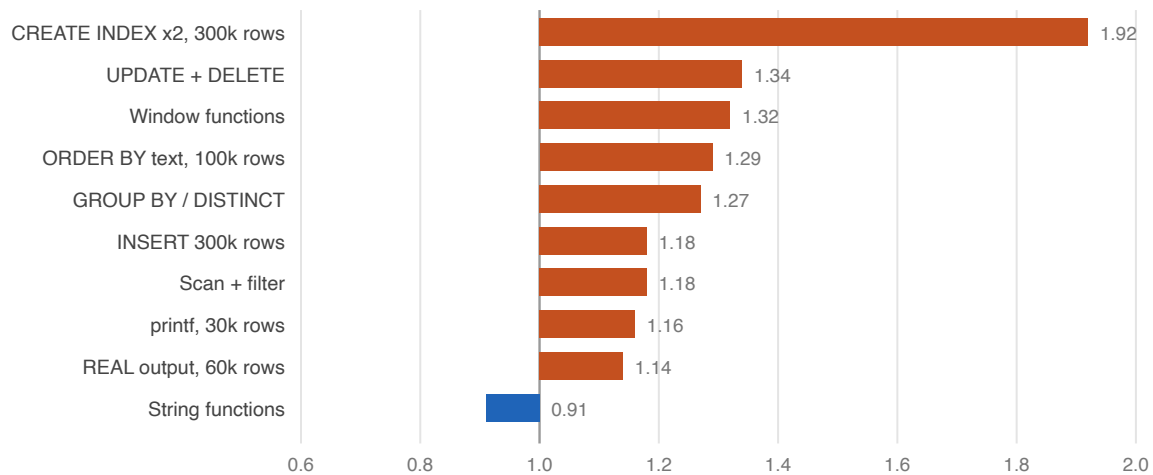


Figure 4. Engine runtime, MoonBit divided by Rust, for ten workloads, mean over three pairs of builds (five runs each; every output matched SQLite).

5 Where MoonBit's Remaining Cost Comes From

Table 5 prices each identifiable source of extra cost at the fitted token prices. The single largest item is the primer, which exists only because current models know MoonBit less well than Rust. Compile errors are second, and helper code that Rust's standard library provides is third. All three are smaller than the saving from writing less code.

Table 5. Estimated cost per build relative to Rust.

Item	Evidence	USD per build
Primer re-reads	11.7k vs 2.9k tokens on each of about 400 calls	+1.39
Language-specific compile errors	Net about 11 calls and 1.9M input tokens	+0.43
Helpers Rust's standard library provides	460–525 lines (about 13k output tokens)	+0.26
Fewer output tokens overall	105k fewer per build, helpers included	−2.10

The helper code is concentrated in a few places. Every MoonBit build wrote its own ordered map (252 to 316 lines) because the core library's sorted map lacked reverse iteration, open and exclusive range bounds, and first and last lookups; the Rust builds used `BTreeMap`. Every build also wrote about 80 lines of exact floating-point digit generation, which Rust provides through its formatting macros, and smaller helpers for checked 64-bit arithmetic, ASCII tests on UTF-16 code units and code-point string comparison.

6 Design Principles for Agent-Oriented Languages

The results suggest that a language's fitness for AI agents rests on two separable factors: how much the agent must write and re-read to express a program, and how often the agent's output fails to compile or behave. MoonBit already does well on the first, which is a property of the design. Its weaknesses are on the second, and they come mostly from the model's knowledge and the library's breadth, which can change. We distill the following principles.

1. **Minimize ceremony per feature.** Every token of ownership annotation, error wrapping and cross-module visibility is written once and re-read on every later call. Garbage collection, effect-typed errors and package-level namespaces removed such tokens without losing static checking.

2. **Make feedback cheap.** A 0.2 s type check let agents validate every edit; MoonBit agents ran moon check 198 times, while Rust agents ran cargo check only 3 times and compiled mostly through full builds.
3. **Keep syntax stable.** Models learn a language from a corpus that lags its current form. Removed constructs were the main reason a primer was needed, and the primer was the largest single overhead.
4. **Ship the data structures agents reach for.** Missing library pieces are rewritten from scratch in every project, and those rewrites were slower than a library implementation would be.
5. **Design diagnostics for filtered output.** Agents piped 368 of 376 build outputs through grep or tail; errors printed before long warning texts were missed four times.
6. **Recognize common foreign idioms.** Constructs borrowed from Rust, such as let (mut a, b), produced cascades of unrelated errors; a targeted message would have cost one turn instead of several.

From measurement to improvement. The first recommendation arising from this study has been implemented: ordered navigation for MoonBit's sorted map (smallest and largest entries, nearest-key queries, ranges with inclusive, exclusive or open bounds, and reverse iteration), together with a shared range-bound type. The change is under review as moonbitlang/core pull request #4333. Because the harness is reproducible, its effect on agent cost can be measured directly by rerunning the benchmark.

Outlook. Three of MoonBit's four overhead items in Table 5 shrink as models gain current MoonBit knowledge and the core library grows; the code-size advantage does not depend on either. If the primer and language-specific errors disappeared and the missing helpers moved into the library, the same arithmetic would place MoonBit at roughly 0.85 to 0.9× Rust's cost on this workload. This is a projection from the measured components, not an observation.

7 Threats to Validity

- **Sample size.** Three builds per language; mean and median disagree on the direction of the cost difference. We make claims of parity, not superiority, on cost.
- **One model and harness.** Results are specific to Claude Opus 5.5 in Claude Code and may differ for other agents.
- **One kind of program.** A single-threaded, data-structure-heavy engine favors garbage-collected languages. Rust's advantages in concurrency, memory control and predictable performance were not exercised, and third-party libraries were excluded by design.
- **Primer design.** A different primer would change MoonBit's results; a no-primer condition is left to future work.
- **Manual labelling.** Compile-failure causes were assigned by hand; the labels are published with the data.
- **Project size.** The engines reached about 13,000 lines rather than the planned 20,000; larger projects may shift the balance between writing and re-reading.

8 Related Work

Function-level benchmarks such as HumanEval [1] and its multilingual translation MultiPL-E [2] measure whether models produce correct code across languages and show that performance tracks the size of each language's training corpus. SWE-bench [3] evaluates agents on real issues in existing Python

repositories. Our study differs in measuring the full cost of constructing a sizable system from scratch, in holding the task fixed while varying only the language, and in attributing cost differences to language and library design.

9 Conclusion

When an AI agent built the same 13,000-line database engine in each language, MoonBit matched Rust on cost, quality and time while producing about 12% smaller programs, despite far less exposure in the model's training data. Its measured overheads are the kind that shrink with time: a primer that compensates for stale model knowledge, a handful of recurring syntax errors, and library gaps, the largest of which is already being addressed. These results support designing languages with agents as a primary user: concise, fast to check, stable in syntax, and complete in the data structures agents need.

Data Availability

The specification, tests, harness, primers, all six engines with per-milestone history, session transcripts, analysis scripts and this paper's figure data are available at <https://github.com/bobzhang/minisql-moonbit-vs-rust>, with a summary at <https://gist.github.com/bobzhang/cbbfc96423a23212d24847a485f1a86d>. The full experiment is rerun with `python3 harness/experiment.py batch --arms rust,moonbit --trials 1-3`.

References

1. M. Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
2. F. Cassano et al. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Transactions on Software Engineering*, 2023.
3. C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR)*, 2024.
4. MoonBit documentation. <https://docs.moonbitlang.com>
5. SQLite Database File Format. <https://www.sqlite.org/fileformat2.html>